Linear Wave Dispersion: Theory, Numerical Methods and Computational Practice

Abstract

The dispersion relation for linear surface gravity waves defines the connection between wave period, wavelength and water depth. Its inversion requires a numerical root or an explicit approximation because the wavenumber appears both as a multiplier and inside a hyperbolic function. This document develops the physical model, derives the dimensionless problem, and provides a complete mathematical catalogue of the 61 methods registered in `wave-disp-equation.py`: one Newton–Raphson reference, 17 standalone approximations, 10 Yamaguchi–Nonaka expressions, 20 Carvalho expressions and 13 fractional rational approximations. Every implemented formula is given, including branch conditions and the complete coefficient sets for the rational family. Accuracy is evaluated on a detailed grid, with separate discussion of numerical error, domain restrictions and timing. A practical manual explains installation, benchmark execution, programmatic use, dimensional calculations and interpretation of the output. Additional wave theories are distinguished from the linear, gravity-only problem solved by the software.

Keywords: surface gravity waves; linear dispersion; explicit inversion; Newton–Raphson; rational approximation; gene expression programming; coastal engineering.

1. Physical formulation

1.1 Scope and assumptions

Consider a progressive wave travelling in the horizontal direction over a flat impermeable seabed. The still-water depth is constant and positive. The fluid is incompressible and inviscid, its motion is irrotational, and atmospheric pressure at the free surface is uniform. Surface tension, mean currents and density stratification are omitted. The wave amplitude is sufficiently small for linearization of the free-surface boundary conditions.

These assumptions define the physical problem. A highly accurate numerical solution of its dispersion equation does not remove the model's physical limitations. Breaking, viscous dissipation, strong nonlinearity, rotational currents and interfacial waves require other formulations.

The source wave-disp-equation.py implements dimensionless scalar solvers and a comparison driver. It neither fits the empirical coefficients nor trains a gene-expression-programming model. The formulas attributed to Carvalho are supplied as fixed expressions; the original fitting datasets and optimization histories are not included.

1.2 Coordinates and primary variables

The horizontal coordinate is x, the vertical coordinate is z, and time is t. The still-water surface is at $z = 0$ and the seabed at $z = -h$. The instantaneous free-surface elevation is $\eta(x, t)$.

Symbol	Meaning	Unit
h	Still-water depth	m
a	Wave amplitude	m
T	Wave period	s
L	Wavelength	m
g	Gravitational acceleration	m/s^2

Symbol	Meaning	Unit
ρ	Fluid density	$\mathrm{kg/m^3}$
k	Angular wavenumber	rad/m
ω	Angular frequency	rad/s

The wave height in the linear monochromatic model is $H = 2a$. Angular wavenumber and angular frequency are defined by:

$$k = \frac{2\pi}{L}, \qquad \omega = \frac{2\pi}{T}$$

Depth must be measured below the water level relevant to the calculation. A negative seabed elevation relative to a vertical datum is not itself a water depth. The dimensional example in Section 9 uses $g = 9.80665\ \mathrm{m/s^2}$; the dimensionless functions have no gravity parameter.

1.3 Potential-flow equations

Irrotationality permits a velocity potential Φ. Incompressibility then leads to Laplace's equation:

$$\mathrm{u} = (u, w) = \nabla\Phi, \qquad \nabla \times \mathrm{u} = 0$$

$$\nabla \cdot \mathrm{u} = 0, \qquad \frac{\partial^2\Phi}{\partial x^2} + \frac{\partial^2\Phi}{\partial z^2} = 0$$

The impermeable seabed imposes zero normal velocity:

$$\left.\frac{\partial\Phi}{\partial z}\right|_{z=-h} = 0$$

At the moving free surface, the exact kinematic condition requires a surface particle to remain on the surface. The dynamic condition follows from Bernoulli's equation after choosing the atmospheric pressure as the pressure reference:

$$\eta_t + \Phi_x\eta_x = \Phi_z \qquad \text{at } z = \eta$$

$$\Phi_t + \frac{1}{2}(\Phi_x^2 + \Phi_z^2) + g\eta = 0 \qquad \text{at } z = \eta$$

Here a subscript denotes partial differentiation. These conditions are nonlinear because they contain products of unknown quantities and are applied at an unknown boundary.

1.4 Linearization and wave solution

When wave steepness and relative amplitude are small, terms of second and higher order in amplitude are neglected and the surface conditions are evaluated at $z = 0$. The formal smallness

requirements include $ka \ll 1$ and $a/h \ll 1$; shallow-water nonlinearity can require additional attention even when steepness is small.

$$\eta_t = \Phi_z, \qquad \Phi_t + g\eta = 0 \qquad \text{at } z = 0$$

A monochromatic progressive solution can be expressed as:

$$\eta(x,t) = a\cos(kx - \omega t)$$

$$\Phi(x,z,t) = A\cosh(k(z+h))\sin(kx - \omega t)$$

The dynamic condition fixes the potential amplitude:

$$A = \frac{ag}{\omega\cosh(kh)}$$

Substitution into the kinematic condition gives the linear gravity-wave dispersion relation:

$$\omega^2 = gk\tanh(kh)$$

The relation is exact within the linear model. Its numerical inversion and the model's physical approximation are distinct sources of uncertainty.

1.5 Derived wavelength and propagation speeds

Once the positive wavenumber is known, wavelength and phase speed follow directly. Differentiation of the dispersion relation at fixed depth gives group speed:

$$L = \frac{2\pi}{k}, \qquad c_p = \frac{\omega}{k} = \frac{L}{T}$$

$$c_p = \frac{g}{\omega}\tanh(kh)$$

$$c_g = \frac{\partial\omega}{\partial k} = \frac{c_p}{2}\left[1 + \frac{2kh}{\sinh(2kh)}\right]$$

The mean energy per unit horizontal area for a linear wave is:

$$E = \frac{1}{2}\rho g a^2 = \frac{1}{8}\rho g H^2$$

At fixed amplitude, this energy expression does not independently depend on wavenumber. Wavenumber nevertheless enters orbital velocities, phase and group speeds, and factors used in wave-energy flux and momentum-flux calculations.

2. Dimensionless inversion problem

2.1 Input and output

Define the deep-water wavenumber and wavelength from frequency alone:

$$k_0 = \frac{\omega^2}{g}, \qquad L_0 = \frac{2\pi}{k_0} = \frac{gT^2}{2\pi}$$

The dimensionless independent variable and unknown are:

$$\alpha = k_0 h = \frac{4\pi^2 h}{gT^2}, \qquad \beta = kh$$

The governing equation becomes:

$$\alpha = \beta \tanh \beta$$

In the code, `k0h` means α, and the returned value means β. Neither value is a dimensional wavenumber. Recover dimensional quantities only after dividing or multiplying by the appropriate depth:

$$k = \frac{\beta}{h}, \qquad L = \frac{2\pi h}{\beta}, \qquad \frac{L}{L_0} = \frac{\alpha}{\beta}$$

For a positive input, the function $\beta \tanh \beta$ increases continuously from zero to infinity. There is therefore exactly one positive root. The case $\alpha = 0$ has the mathematical limiting solution $\beta = 0$, but not every implemented expression evaluates that limit without a special guard.

2.2 Shallow- and deep-water limits

For small dimensionless wavenumber, the hyperbolic tangent has the expansion:

$$\tanh \beta = \beta - \frac{\beta^3}{3} + \frac{2\beta^5}{15} + \cdots$$

Consequently:

$$\alpha \sim \beta^2, \qquad \beta \sim \sqrt{\alpha}, \qquad k \sim \frac{\omega}{\sqrt{gh}}$$

$$L \sim T\sqrt{gh}, \qquad c_p \sim c_g \sim \sqrt{gh}$$

For large dimensionless wavenumber, the hyperbolic tangent approaches one:

$$\tanh \beta \to 1, \qquad \beta \sim \alpha, \qquad k \sim k_0, \qquad L \sim L_0$$

$$c_p \sim \frac{g}{\omega} = \frac{gT}{2\pi}, \qquad c_g \sim \frac{c_p}{2}$$

The common practical depth classifications use the local wavelength: shallow water is approximately $h/L < 0.05$ and deep water approximately $h/L > 0.5$. They are convenient classifications rather than abrupt physical transitions. Ratios involving L_0 must not be substituted for ratios involving L without accounting for their difference.

2.3 Error variables

Let β_r denote the numerical reference and β_a an approximation. For fixed input period and depth:

$$\delta_\beta = \frac{\beta_a - \beta_r}{\beta_r} = \frac{k_a - k_r}{k_r} = \delta_k$$

Wavelength and phase speed depend on the reciprocal wavenumber:

$$\delta_L = \delta_{c_p} = -\frac{\delta_k}{1 + \delta_k}$$

Thus signed wavelength and wavenumber errors have opposite signs and slightly different magnitudes. Absolute percentages quoted for different quantities are not interchangeable. For a phase-resolving calculation over a distance x, the phase discrepancy is:

$$\Delta\theta = x\Delta k$$

These relationships help select an accuracy requirement appropriate to the application. A sampled percentage error alone does not establish whether an approximation is adequate for every propagation distance or model.

3. Numerical reference and correction operator

3.1 Newton–Raphson method

The reference function is `kh_numeric(k0h, tol=1e-15, max_iter=100)`. It solves the residual equation:

$$f(\beta) = \alpha - \beta \tanh \beta = 0$$

$$f'(\beta) = -\tanh \beta - \frac{\beta}{\cosh^2 \beta}$$

Newton's update is:

$$\beta_{n+1} = \beta_n - \frac{\alpha - \beta_n \tanh \beta_n}{-\tanh \beta_n - \beta_n / \cosh^2 \beta_n}$$

The first derivative is strictly negative for positive β. The residual has an inflection near $\beta = 1.200$, but this is not a horizontal tangent and does not itself imply Newton divergence. The second derivative makes the distinction explicit:

$$f''(\beta) = \frac{2}{\cosh^2 \beta}(\beta \tanh \beta - 1)$$

3.2 Initial estimate and stopping rule

The implementation uses the Carvalho-style initial estimate:

$$\beta_0 = \frac{\alpha}{\tanh((6/5)^\alpha \sqrt{\alpha})}$$

This is also Carvalho formula 10 in Section 6. It has the correct leading shallow- and deep-water limits and gives a close starting value on the comparison interval.

At each step, the implementation calculates $\Delta\beta = f/f'$, subtracts it from the current estimate, and terminates when:

$$\left|\frac{\Delta\beta}{\beta_n}\right| < 10^{-15}$$

The tolerance is a relative correction criterion, not a rigorous bound on the root error. If the maximum iteration count is reached, the function returns the last estimate without raising a convergence exception. At zero input it returns zero immediately. Negative and non-finite inputs are not comprehensively validated.

The word "exact" in some source comments means this numerical reference. It does not mean a symbolic solution, arbitrary precision, or an independently certified floating-point result.

3.3 A reusable one-step correction

Several registered approximations consist of an explicit seed followed by one Newton correction. Define the operator:

$$\mathcal{N}(b; \alpha) = b - \frac{\alpha - b \tanh b}{-\tanh b - b/\cosh^2 b}$$

Equivalent forms useful for checking the implementations are:

$$\mathcal{N}(b; \alpha) = \frac{\alpha + b^2/\cosh^2 b}{\tanh b + b/\cosh^2 b}$$

$$\mathcal{N}(b; \alpha) = \frac{b^2 + \alpha \cosh^2 b}{b + \sinh b \cosh b}$$

The equivalence is algebraic. Floating-point overflow and rounding can differ between forms. A fixed single correction is an approximation; it does not apply a convergence test and need not achieve machine precision.

3.4 Notation for the formula catalogue

Every expression below returns an approximation to $\beta = kh$ for a positive α. The symbols B, Y_j, C_j and P_j denote particular method outputs, not new independent physical variables. Hyperbolic cotangent means $\coth x = 1/\tanh x$.

The catalogue specifies the implemented coefficient values and branch thresholds. Method years and spellings identify registered code labels; historical attribution is discussed separately in the bibliography. Algebraically identical methods remain listed because they appear as separate entries in the benchmark.

4. Standalone explicit approximations

4.1 Eckart

Function: `eckart1951`. The formula joins the leading shallow- and deep-water limits using one hyperbolic tangent:

$$\beta_E = \frac{\alpha}{\sqrt{\tanh \alpha}}$$

Equivalently, its approximate wavelength is:

$$L_E = L_0 \sqrt{\tanh \alpha}$$

Its small expression size is useful for comparison, although its maximum sampled error is larger than that of the more refined methods.

4.2 Iwagaki

Function: `iwagaki2007`. The source describes an Iwagaki expression reproduced in the Yamaguchi–Nonaka comparison:

$$\beta_I = \frac{\alpha}{\tanh\left[\sqrt{\alpha}\left(1 + \frac{\sqrt{\alpha}}{2\pi}\right)\right]}$$

This is identical to Yamaguchi formula 3 in the implementation.

4.3 Fenton and McKee

Function `fenton_mckee1990_1` applies the correction operator to the Eckart seed:

$$b = \frac{\alpha}{\sqrt{\tanh \alpha}}, \qquad \beta_{FM1} = \mathcal{N}(b; \alpha)$$

Its implemented numerator and denominator are:

$$\beta_{FM1} = \frac{\alpha + b^2 \operatorname{sech}^2 b}{\tanh b + b \operatorname{sech}^2 b}$$

Function `fenton_mckee1990_2` is the generalized hyperbolic-power approximation with exponent parameter $m = 1.5$:

$$\beta_{FM2} = \frac{\alpha}{[\tanh(\alpha^{3/4})]^{2/3}}$$

The underlying parametric form is:

$$F_m(\alpha) = \alpha[\coth(\alpha^{m/2})]^{1/m}$$

Other fitted values of m appear in the Yamaguchi family. The two Fenton–McKee code functions are therefore distinct computational constructions, not two names for the same expression.

4.4 Hunt: fifth-order denominator

Function: `hunt1979_5`.

$$\beta_{H5} = \left[\alpha^2 + \frac{\alpha}{D_5(\alpha)}\right]^{1/2}$$

$$D_5(\alpha) = 1 + 0.6522\alpha + 0.4622\alpha^2 + 0.0864\alpha^4 + 0.0675\alpha^5$$

There is no cubic term in this implemented denominator. The final square root is essential; the rational expression alone gives the square of the dimensionless wavenumber.

4.5 Hunt: ninth-order denominator

Function: `hunt1979_9`.

$$\beta_{H9} = \left[\alpha^2 + \frac{\alpha}{1 + \sum_{j=1}^{9} d_j \alpha^j}\right]^{1/2}$$

The full coefficient set below follows the source, including the seventh and ninth coefficients.

j	d_j
1	0.6666666667
2	0.3555
3	0.16084
4	0.0632
5	0.02174
6	0.00654
7	0.00171
8	0.00039
9	0.00011

The Python code evaluates a sum of powers and a square root. A Horner implementation could reorganize the arithmetic, but that is not the evaluation sequence used in this version.

4.6 Wu and Thornton

Function: `wu_thornton1986`. The switching point is $\alpha = 0.4\pi$, corresponding to $h/L_0 = 0.2$.

For $\alpha \leq 0.4\pi$:

$$\beta_{WT} = \sqrt{\alpha}\left[1 + \frac{\alpha}{6}\left(1 + \frac{\alpha}{5}\right)\right]$$

For $\alpha > 0.4\pi$, define:

$$y = \alpha[1 + 1.26\exp(-1.84\alpha)]$$

$$\beta_{WT} = \alpha[1 + 2\exp(-2y)(1 + \exp(-2y))]$$

The branch decision is part of the formula. A different switching point changes the implemented approximation and its error statistics.

4.7 Nielsen and You

Functions: `nielsen1982` and `you2002`. These two functions implement exactly the same piecewise expression.

For $\alpha \leq 2$:

$$\beta_{NY} = \sqrt{\alpha}\left(1 + \frac{\alpha}{3} + \frac{4\alpha^2}{45} + \frac{16\alpha^3}{945}\right)^{1/2}$$

For $\alpha > 2$:

$$\beta_{NY} = \alpha[1 + 2\exp(-2\alpha)]$$

They therefore have the same calculated accuracy, apart from possible formatting or measurement differences. Their distinct labels preserve the source's organization and associated references.

4.8 Gilbert

Function: `gilbert2000`. For $\alpha \leq 1$:

$$\beta_G = \sqrt{\alpha}(1 + 0.2\alpha)$$

For $\alpha > 1$:

$$\beta_G = \alpha[1 + 0.2\exp(2 - 2\alpha)]$$

Both expressions give 1.2 at the switch. This continuity does not by itself establish continuity of derivatives or a global accuracy bound.

4.9 Guo

Function: `guo2002`.

$$\beta_{Guo} = \frac{\alpha}{[1 - \exp(-\alpha^{m/2})]^{1/m}}, \qquad m = 2.4901$$

The fitted parameter must be preserved. The Yamaguchi family contains a related expression with a different value of m.

4.10 Guan and Ju

Function: `guan2005`.

$$\beta_{GJ} = \sqrt{\alpha}\exp(-1.115\alpha) + \alpha\tanh(1.325\sqrt{\alpha})$$

The first term contributes principally at small input and the second approaches the deep-water behaviour as input increases.

4.11 Beji

Function: `beji2013`. Define the Beji estimate $B(\alpha)$ by:

$$B(\alpha) = \frac{\alpha}{\sqrt{\tanh\alpha}}[1 + \alpha^{1.09}\exp(-1.55 - 1.30\alpha - 0.216\alpha^2)]$$

The expression combines the Eckart denominator with an additive exponential correction.

4.12 Vatankhah and Aghashariatmadari

Function `vatankhah2013_2` implements the shorter exponential correction:

$$\beta_{V2} = \frac{\alpha + \alpha^2\exp(-1.835 - 1.225\alpha^{1.35})}{\sqrt{\tanh\alpha}}$$

Function `vatankhah2013_1` implements a sum of two terms:

$$\beta_{V1} = A_V(\alpha) + B_V(\alpha)$$

$$A_V(\alpha) = \frac{\alpha + \alpha^2\exp[-(3.2 + \alpha^{1.65})]}{\sqrt{\tanh\alpha}}$$

$$B_V(\alpha) = \alpha[1 - \exp(-\alpha^{0.132})]^{5.0532 + 2.1584\alpha^{1.505}}$$

The function names and registered labels distinguish these two expressions: suffix 1 denotes the split expression, while suffix 2 denotes the shorter exponential correction.

4.13 Simarro and Orfila

Function: `Simarro_2013`. The Beji estimate is followed by one Newton correction:

$$\beta_{SO} = \mathcal{N}(B(\alpha); \alpha)$$

Writing $b = B(\alpha)$ gives the expression evaluated by the function:

$$\beta_{SO} = \frac{b^2 + \alpha \cosh^2 b}{b + \sinh b \cosh b}$$

The resulting error is much smaller than the seed error on the comparison grid. It remains a finite approximation error and must not be equated with symbolic exactness or guaranteed machine precision.

4.14 Yu

Function: `yu2014`.

$$q = 2[\tanh \alpha]^{2.5} - 1$$

$$\beta_{Yu} = \frac{\alpha}{\sqrt{\tanh \alpha}} + 0.0527 \sin(\arccos q)$$

For a positive real input, the mathematical argument q lies in the interval from minus one to one. The source evaluates the inverse cosine directly, without a separate clipping operation.

5. Yamaguchi–Nonaka family: all ten expressions

The selector call is `YamaguchiNonaka(k0h, formula)`, where `formula` is an integer from 1 to 10. Denote the resulting approximation by $Y_j(\alpha)$. A zero-input guard returns zero before the selector is evaluated. For positive input, an unsupported selector returns the sentinel value `-1`.

5.1 Generalized hyperbolic forms: formulas 1 and 5

Using F_m from Section 4.3:

$$Y_1(\alpha) = F_{1.485}(\alpha) = \alpha[\coth(\alpha^{1.485/2})]^{1/1.485}$$

$$Y_5(\alpha) = F_{1.434}(\alpha) = \alpha[\coth(\alpha^{1.434/2})]^{1/1.434}$$

5.2 Nested substitution: formulas 2 and 8

The initial fitted hyperbolic estimate is substituted into a rearrangement of the dispersion relation:

$$Y_2(\alpha) = \frac{\alpha}{\tanh[F_{1.378}(\alpha)]}$$

$$Y_8(\alpha) = \frac{\alpha}{\tanh[F_{1.310}(\alpha)]}$$

These are fixed substitutions, not Newton corrections. Their internal parameter values are part of the fitted expressions.

5.3 Iwagaki and algebraic forms: formulas 3 and 4

$$Y_3(\alpha) = \frac{\alpha}{\tanh[\sqrt{\alpha}(1 + \sqrt{\alpha}/(2\pi))]}$$

$$Y_4(\alpha) = \alpha(1 + \alpha^{-2})^{1/4}$$

Formula 3 duplicates `iwagaki2007`. Formula 4 uses algebraic powers only, but can still be sensitive to intermediate overflow or underflow at extreme inputs.

5.4 Hyperbolic and exponential forms: formulas 6, 7 and 9

$$Y_6(\alpha) = \frac{\alpha}{\tanh(\sqrt{\sinh \alpha})}$$

$$Y_7(\alpha) = \frac{\alpha}{[1 - \exp(-\alpha^{2.445/2})]^{1/2.445}}$$

$$Y_9(\alpha) = \frac{\alpha}{\tanh(1.1965^{\alpha}\sqrt{\alpha})}$$

The differences between 2.445 and Guo's 2.4901, and between 1.1965 and Carvalho's 1.2 base, must be retained when reproducing the benchmark.

5.5 Nested hyperbolic expression: formula 10

Define a common denominator factor:

$$Q(\alpha) = [\tanh \alpha]^{1/4}\sqrt{\tanh(\sqrt{\sinh \alpha})}$$

Then:

$$Y_{10}(\alpha) = \frac{\alpha}{\tanh[\alpha/Q(\alpha)]}$$

This duplicates Carvalho formula 3. The common factor Q is also used by Carvalho formulas 1 and 9.

6. Carvalho family: all twenty expressions

The selector call is `carvalho2025(k0h, formula)`, with an integer selector from 1 to 20. The notation C_j below corresponds exactly to the source selector. Unlike several standalone routines, the family has no common zero-input guard. The formulas in this section are therefore stated for positive input.

6.1 Corrected nested estimate: formula 1

The seed is Carvalho formula 3 and the final result uses the correction operator:

$$b = C_3(\alpha) = \frac{\alpha}{\tanh[\alpha/Q(\alpha)]}$$

$$C_1(\alpha) = \mathcal{N}(b;\alpha) = \frac{b^2 + \alpha\cosh^2 b}{b + \sinh b \cosh b}$$

This distinguishes formula 1 from an uncorrected explicit estimate. Its cost includes both the nested seed and the correction.

6.2 Piecewise fitted approximation: formula 2

For $0 < \alpha \leq 1.2$:

$$C_2(\alpha) = \frac{\sqrt{1/\alpha - \exp(\alpha^{1.962983} - 6.242035)}}{1/\alpha - 0.168659434}$$

For $1.2 < \alpha \leq 2.35$, define two shorter expressions:

$$U_2 = \alpha + (\alpha/70.13327717)^{\alpha^3}$$

$$V_2 = \exp\left[\frac{\ln(4.89859^{\alpha})}{1.134674 - 10^{\alpha}}\right], \qquad C_2(\alpha) = \frac{U_2}{V_2}$$

For $\alpha > 2.35$:

$$C_2(\alpha) = \alpha \exp\left(\frac{1.596671172\alpha}{10^{\alpha}}\right)$$

Both the thresholds and the complete decimal coefficients are required. Continuity near the switches and behaviour outside the comparison interval should be assessed separately when the formula is reused.

6.3 Nested hyperbolic approximations: formulas 3, 4 and 7

$$C_3(\alpha) = \frac{\alpha}{\tanh[\alpha/Q(\alpha)]} = Y_{10}(\alpha)$$

For formula 4, a sequence of intermediate quantities avoids a long nested fraction while preserving every operation:

$$r_0 = \sinh(\tanh\sqrt{\alpha}), \qquad r_1 = \tanh(\alpha/r_0)$$

$$r_2 = \tanh(\alpha/r_1), \qquad r_3 = \tanh(\alpha/r_2)$$

$$C_4(\alpha) = \frac{\alpha}{r_3}$$

Formula 7 uses one fewer nested level:

$$C_7(\alpha) = \frac{\alpha}{r_2}$$

The r_j quantities are temporary dimensionless values. They are not iteration states of a convergence loop.

6.4 Fitted exponential bridges: formulas 5 and 6

$$C_5(\alpha) = \frac{\alpha}{\tanh[1.199315^{\alpha^{1.047086}}\alpha^{0.499947}]}$$

$$C_6(\alpha) = \frac{\alpha}{\tanh[1.1999^{\alpha^{1.045}}\sqrt{\alpha}]}$$

Formula 5 can also be written directly for wavelength:

$$L_{C5} = L_0 \tanh[1.199315^{\alpha^{1.047086}}\alpha^{0.499947}]$$

The exponent 0.499947 is deliberately different from 0.5. Its small difference matters for the fitted curve and means that the leading mathematical power is not exactly the shallow-water power in the limit of arbitrarily small input. Good accuracy on a finite interval does not establish exact asymptotic matching.

6.5 Capped intermediate argument: formula 8

$$a_c = \min(\alpha, 3.04425)$$

$$C_8(\alpha) = \frac{\alpha}{\tanh[\sinh(\sqrt{a_c})\cosh(\alpha/5.194671)]}$$

The cap applies only inside the square root and hyperbolic sine; the remaining appearances of α use the original input.

6.6 Hyperbolic product and exponential seeds: formulas 9–11

$$C_9(\alpha) = \frac{\alpha}{Q(\alpha)}$$

$$C_{10}(\alpha) = \frac{\alpha}{\tanh[(6/5)^{\alpha}\sqrt{\alpha}]}$$

$$C_{11}(\alpha) = \frac{\alpha}{\tanh[\sqrt{1.438995^{\alpha}\alpha}]}$$

Formula 10 is the seed of the iterative reference. Formula 9 is an uncorrected product-denominator approximation, whereas formula 3 adds an outer hyperbolic substitution to it.

6.7 Nested and additive forms: formulas 12 and 13

$$C_{12}(\alpha) = \frac{\alpha}{\tanh\left[\frac{\alpha}{\tanh(\sinh\sqrt{\alpha})}\right]}$$

$$C_{13}(\alpha) = \alpha + \frac{\sqrt{\alpha}}{4.35144^{\alpha} + 0.718409/(1/\alpha)^{0.437408}}$$

The second denominator term in formula 13 is algebraically equal to $0.718409\alpha^{0.437408}$ for positive input. The form above follows the source evaluation.

6.8 Variable-power and product forms: formulas 14–17

$$C_{14}(\alpha) = \frac{\alpha}{[\tanh\sqrt{\alpha}]^{1/\cosh\alpha}}$$

$$C_{15}(\alpha) = \frac{\alpha}{\sqrt{\tanh\alpha}\cdot\tanh(\alpha + 1/\sqrt{\alpha})}$$

$$C_{16}(\alpha) = \frac{\alpha}{[\tanh\alpha]^{(\alpha+4)/8}}$$

$$C_{17}(\alpha) = \frac{\alpha}{\sqrt{(\tanh\alpha)^{\alpha/\tanh\alpha}}}$$

6.9 Remaining compact expressions: formulas 18–20

$$C_{18}(\alpha) = \frac{\alpha}{\tanh(\sinh\sqrt{\alpha})}$$

$$C_{19}(\alpha) = \sqrt{\alpha} + \frac{\alpha^2}{\alpha + 4}$$

$$C_{20}(\alpha) = \frac{\alpha}{\sqrt[4]{(\sqrt{\tanh\alpha})^{\tanh\alpha+4}}}$$

Different algebraic constructions can produce similar maximum errors but different distributions of error across depth. The full benchmark is more informative than a ranking based only on expression length.

7. Fractional rational family: all thirteen coefficient sets

7.1 General representation

The selector call is `pade2025(k0h, formula)`, with selectors 1–13. The source calls these Padé approximants. A classical Padé construction matches a Taylor expansion to a specified order; the code provides fitted coefficients but not a derivation establishing that matching property. The term **fractional rational approximation** describes the implemented form without requiring that additional claim.

For selector r, let $m = r + 1$. Every expression can be written as:

$$P_r(\alpha) = \sqrt{\alpha}\,\frac{A_r(\alpha)}{B_r(\alpha)}$$

$$A_r(\alpha) = \sum_{j=0}^{m} a_j \alpha^j, \qquad B_r(\alpha) = \sum_{j=0}^{m} b_j \alpha^j, \qquad b_0 = 1$$

The tables below provide **all coefficients**, without omitted terms or rounded shorthand. An entry written in scientific notation retains the numeric literal used in the source. For example, a coefficient shown as `8.4e-5` would mean 8.4×10^{-5}. The actual coefficients below are not reduced to that example's precision.

7.2 Domain and asymptotic behaviour

The function documentation describes an intended interval from zero to 2π, but the function does not enforce that interval. Unsupported selectors return `-1`. At zero input, the implemented valid rational expressions evaluate to zero; this does not eliminate the need to check the denominator and conditioning elsewhere.

For small positive input:

$$P_r(\alpha) \sim a_0\sqrt{\alpha}$$

The leading coefficient is close to one, but generally not exactly one. For large input, when the displayed leading numerator and denominator coefficients are nonzero:

$$P_r(\alpha) \sim \frac{a_m}{b_m}\sqrt{\alpha}$$

This is not the exact deep-water behaviour $\beta \sim \alpha$. Consequently, excellent agreement within the fitted interval must not be interpreted as permission for unrestricted deep-water extrapolation. Higher polynomial degree alone does not guarantee monotonic improvement in maximum error or numerical conditioning.

7.3 Formula 1: `Pade(2025)_1`

$$P_1(\alpha) = \sqrt{\alpha}\,\frac{\sum_{j=0}^{2} a_j \alpha^j}{\sum_{j=0}^{2} b_j \alpha^j}$$

j	a_j	b_j
0	1.00649052194019	1
1	0.423646282789217	0.306955955676234
2	0.175406661440005	0.0328975279727171

7.4 Formula 2: `Pade(2025)_2`

$$P_2(\alpha) = \sqrt{\alpha}\,\frac{\sum_{j=0}^{3} a_j \alpha^j}{\sum_{j=0}^{3} b_j \alpha^j}$$

j	a_j	b_j
0	0.998980252114366	1
1	0.0240176797055886	-0.150350405960952
2	0.102524886754552	0.112157962910113
3	0.0317327085938995	0.00294483072586115

7.5 Formula 3: `Pade(2025)_3`

$$P_3(\alpha) = \sqrt{\alpha}\,\frac{\sum_{j=0}^{4} a_j \alpha^j}{\sum_{j=0}^{4} b_j \alpha^j}$$

j	a_j	b_j
0	1.00006668638419	1
1	0.322645945302282	0.157166943736625
2	0.0860384450810725	0.0245168267924732
3	0.051143347041175	0.0462567432956417
4	0.0153420957423937	0.00175392506101448

7.6 Formula 4: `Pade(2025)_4`

$$P_4(\alpha) = \sqrt{\alpha}\,\frac{\sum_{j=0}^{5} a_j \alpha^j}{\sum_{j=0}^{5} b_j \alpha^j}$$

j	a_j	b_j
0	0.999996682596798	1
1	-0.0889915717930786	-0.255723982020183
2	0.147076211695128	0.159493904911975
3	0.0123471280480147	-0.0106101311382749
4	0.00866458140843225	0.00784491418150148
5	0.00204463718201973	0.000184273251439305

7.7 Formula 5: `Pade(2025)_5`

$$P_5(\alpha) = \sqrt{\alpha}\,\frac{\sum_{j=0}^{6} a_j \alpha^j}{\sum_{j=0}^{6} b_j \alpha^j}$$

j	a_j	b_j
0	0.999998218345888	1
1	-0.424362176674708	-0.591069429462395
2	0.171875463304611	0.240083348894323
3	-0.0357487982640122	-0.0617593442909405
4	0.00410625374333464	0.0104920694265126
5	-0.000978753693904127	-0.00231970889331938
6	-0.000636955605769902	-5.65924775627923e-5

7.8 Formula 6: `Pade(2025)_6`

$$P_6(\alpha) = \sqrt{\alpha}\,\frac{\sum_{j=0}^{7} a_j \alpha^j}{\sum_{j=0}^{7} b_j \alpha^j}$$

j	a_j	b_j
0	1.0000000012405	1
1	-0.350251200743747	-0.516917882097882
2	0.229153326540668	0.284751410622371
3	-0.0205204312544928	-0.0555622365621819
4	0.0133231478358294	0.0161071584333013
5	0.0010401274983046	-0.000808341017586247
6	0.00048671850792775	0.000381511960690599
7	8.40088474488992e-5	6.40735447518177e-6

7.9 Formula 7: `Pade(2025)_7`

$$P_7(\alpha) = \sqrt{\alpha}\,\frac{\sum_{j=0}^{8} a_j \alpha^j}{\sum_{j=0}^{8} b_j \alpha^j}$$

j	a_j	b_j
0	0.999999995257458	1
1	-0.543811114837314	-0.710477969437912
2	0.297774393256421	0.385633880896532
3	-0.0648661921727468	-0.110812474410256
4	0.0174768559302056	0.0270500418196004
5	-0.00151793039097231	-0.00394602590941703
6	0.000295750461715408	0.000553921270262525
7	-4.1567697098083e-6	-6.65533846723705e-5

j	a_j	b_j
8	-1.62498860684328e-5	-1.24656671282763e-6

7.10 Formula 8: `Pade(2025)_8`

$$P_8(\alpha) = \sqrt{\alpha}\,\frac{\sum_{j=0}^{9} a_j \alpha^j}{\sum_{j=0}^{9} b_j \alpha^j}$$

j	a_j	b_j
0	1.00000000020126	1
1	-0.388439115555858	-0.555105771884377
2	0.310223332529737	0.372185262898829
3	-0.0496321949056331	-0.0980736559689507
4	0.0293825301580729	0.036689986163131
5	-0.000149900084396432	-0.00440840853967443
6	0.00139739652490532	0.00139722437126806
7	0.000171592322622253	-3.64106868131082e-6
8	4.94349555930422e-5	2.72315576473091e-5
9	5.36329658499187e-6	3.80576928768955e-7

7.11 Formula 9: `Pade(2025)_9`

$$P_9(\alpha) = \sqrt{\alpha}\,\frac{\sum_{j=0}^{10} a_j \alpha^j}{\sum_{j=0}^{10} b_j \alpha^j}$$

j	a_j	b_j
0	1.00000000054683	1
1	-0.302517970258141	-0.469184609052338
2	0.216194173804703	0.263835664763448
3	-0.00911867675112112	-0.0421256860069538
4	0.0131923444114312	0.0141905686336732
5	0.00197230469011907	0.000171234247175628
6	0.000583685774943952	0.000284037180073895
7	0.000117061426950459	7.23728845968762e-5
8	2.16577683514269e-5	-3.33925175755566e-6
9	-4.06436904033371e-6	-1.1196042731312e-6
10	-1.35655741732812e-7	-6.10361897619335e-9

7.12 Formula 10: `Pade(2025)_10`

$$P_10(\alpha) = \sqrt{\alpha}\,\frac{\sum_{j=0}^{11} a_j \alpha^j}{\sum_{j=0}^{11} b_j \alpha^j}$$

j	a_j	b_j
0	1.00000000069543	1
1	-0.334954381847524	-0.501621014271701
2	0.229997336203832	0.283044819037831
3	-0.0170932674073579	-0.0523102821262626
4	0.0142286437913579	0.0164455383396673
5	0.00157766182461221	-0.000365622576684072
6	0.000555414435408452	0.000304882740985283
7	0.000109241385068583	7.15596032735346e-5
8	2.04181414663277e-5	-5.64260090297976e-6
9	-4.47936721320148e-6	-7.22847827757772e-7
10	7.20245847805242e-8	3.16954475149092e-8
11	2.27591359161482e-9	-4.18523172095159e-11

7.13 Formula 11: `Pade(2025)_11`

$$P_1 1(\alpha) = \sqrt{\alpha}\,\frac{\sum_{j=0}^{12} a_j \alpha^j}{\sum_{j=0}^{12} b_j \alpha^j}$$

j	a_j	b_j
0	1.00000000021134	1
1	-0.439538511010958	-0.606205166733527
2	0.262071966075091	0.332550449569891
3	-0.0352260757847662	-0.0755002415059252
4	0.0136888861362354	0.0186169006337531
5	0.00119983590894612	-0.000624409536143469
6	0.000306132196963262	0.000110698204767146
7	9.3344593984067e-5	7.31839474973857e-5
8	1.13485236045952e-5	-4.17221912299122e-6
9	-2.19033671564094e-6	-1.00985652913037e-6
10	-1.52303393432862e-7	1.15600808535017e-7
11	3.29680588537e-8	-7.31093937337407e-9
12	-3.59648926971857e-9	-3.576922099426e-10

7.14 Formula 12: `Pade(2025)_12`

$$P_1 2(\alpha) = \sqrt{\alpha}\,\frac{\sum_{j=0}^{13} a_j \alpha^j}{\sum_{j=0}^{13} b_j \alpha^j}$$

j	a_j	b_j
0	1.00000000034658	1
1	-0.35765423836608	-0.524320887192481
2	0.220474157851537	0.277305164506462
3	-0.0143101981637556	-0.0478782214771677

j	a_j	b_j
4	0.0106882664277003	0.0124223110237922
5	0.00177809831685665	0.00037129622072912
6	0.000425551341246499	0.000158246646088704
7	8.39454668343144e-5	5.24876551697954e-5
8	1.68918586510378e-5	-1.38557325635002e-6
9	-3.47736023040874e-6	-7.96232034767084e-7
10	1.1564308170262e-7	8.35464703802028e-9
11	-5.38824115168048e-9	1.1834526426828e-8
12	2.55400838802905e-9	-1.48873774124982e-9
13	-5.77246087402385e-10	-5.59846037648501e-11

7.15 Formula 13: `Pade(2025)_13`

$$P_13(\alpha) = \sqrt{\alpha}\,\frac{\sum_{j=0}^{14} a_j \alpha^j}{\sum_{j=0}^{14} b_j \alpha^j}$$

j	a_j	b_j
0	1.00000000043044	1
1	-0.341214787680155	-0.507881432557414
2	0.216029315236116	0.27012036160695
3	-0.0116443516054976	-0.0445169260696995
4	0.0108812744435703	0.0122182321499095
5	0.00184089184824533	0.00039913233839911
6	0.000466873470691525	0.000190210573213751
7	8.83178779786945e-5	5.3696821111172e-5
8	1.82969284264395e-5	-1.69377956737975e-6
9	-3.70984687913519e-6	-7.45889615784955e-7
10	1.15252770559743e-7	1.5632250659559e-9
11	4.05866020101675e-10	1.19811786331838e-8
12	1.00682972256747e-9	-1.3440816876962e-9
13	-1.03386709606535e-10	1.13151579925971e-10
14	2.54165346162395e-11	1.46210486272321e-12

8. Comparative evaluation

8.1 Sampling and error calculation

The benchmark uses 10,000 equally spaced values, including the endpoints, on $0.0001 \le \alpha \le 2\pi$. Every method is evaluated on the same grid and compared against `kh_numeric`. The mean and maximum absolute relative errors are calculated in percent:

$$e_i = 100 \left| \frac{\beta_{a,i} - \beta_{r,i}}{\beta_{r,i}} \right|$$

$$\overline{e} = \frac{1}{N}\sum_{i=1}^{N} e_i, \qquad e_{\max} = \max_{1 \le i \le N} e_i$$

The input associated with the maximum is the first sampled input attaining it. Mean error depends on the sampling distribution; uniform sampling in α is not equivalent to uniform sampling in period, depth or logarithmic relative depth. A sampled maximum is not a proven supremum over a continuous interval.

`compute_errors` assigns zero error if the reference value is zero. It cannot therefore establish the correctness of a method's zero-input limit. The default comparison avoids that issue by starting at 0.0001.

8.2 Complete accuracy results

The following results are calculated on the default grid using the implementation. Rows are sorted by mean error, then by maximum error, using unrounded values. Scientific notation is retained to distinguish very small errors from exact zero. The numerical reference has zero error against itself by definition.

Method	Mean error (%)	Maximum error (%)
kh_numeric	0	0
Pade(2025)_11	8.766532e-10	2.102563e-08
Pade(2025)_8	1.119506e-09	2.002354e-08
Pade(2025)_12	1.275911e-09	3.44811e-08
Pade(2025)_13	1.457029e-09	4.282786e-08
Pade(2025)_9	2.144098e-09	5.440632e-08
Pade(2025)_10	2.299322e-09	6.920335e-08
Pade(2025)_7	2.611842e-08	7.486189e-06
Pade(2025)_6	3.786279e-08	1.535453e-07
Carvalho(2025)_1	3.050183e-07	4.234258e-06
Simarro&Orfila(2013)	4.898401e-07	8.223883e-06
Pade(2025)_4	1.360187e-05	0.0003310862
Pade(2025)_5	1.862044e-05	0.006444387
Vatankhah(2013)_1	0.0004900532	0.001758975
Pade(2025)_3	0.0006188451	0.006656551
Carvalho(2025)_2	0.002832872	0.01672207
Hunt(1979)_9	0.003576223	0.008150303
Fenton&McKee(1990)_1	0.005598106	0.05111372
Pade(2025)_2	0.005928564	0.1018976
Wu&Thornton(1986)	0.006984842	0.03427385
Vatankhah(2013)_2	0.008544918	0.01889016
Carvalho(2025)_3	0.009439052	0.04446302
Yamaguchi(2007)_10	0.009439052	0.04446302
Carvalho(2025)_4	0.009797759	0.05014884
Beji(2013)	0.01641125	0.04422621
Carvalho(2025)_5	0.02935249	0.07602468
Carvalho(2025)_6	0.02960853	0.06541103

Method	Mean error (%)	Maximum error (%)
Carvalho(2025)_7	0.03089261	0.1351853
Hunt(1979)_5	0.03858343	0.07837798
Nielsen(1982)	0.04512237	0.5804859
You(2002)	0.04512237	0.5804859
Carvalho(2025)_8	0.06424006	0.1698012
Carvalho(2025)_9	0.06685411	0.2037817
Carvalho(2025)_10	0.07297119	0.271227
Carvalho(2025)_11	0.07425218	0.2609919
Yu(2014)	0.08639421	0.3291155
Yamaguchi(2007)_9	0.08839415	0.3122414
Carvalho(2025)_12	0.08923112	0.4009798
Pade(2025)_1	0.1226442	0.6485218
Carvalho(2025)_13	0.1267865	0.3714239
Yamaguchi(2007)_2	0.1888776	0.7321854
Carvalho(2025)_14	0.2085362	1.278469
Gilbert(2000)	0.2292764	0.7930431
Yamaguchi(2007)_8	0.2554179	1.109223
Guo(2002)	0.2863639	0.7571934
Carvalho(2025)_15	0.2925759	1.311094
Carvalho(2025)_16	0.3101789	1.421656
Guan&Ju(2005)	0.3393654	0.4970393
Carvalho(2025)_17	0.3571345	1.83103
Yamaguchi(2007)_7	0.3597263	1.031363
Carvalho(2025)_18	0.3724391	1.12898
Carvalho(2025)_19	0.4540868	1.389263
Carvalho(2025)_20	0.5191037	2.023581
Fenton&McKee(1990)_2	0.6716352	1.631425
Yamaguchi(2007)_1	0.7077884	1.542839
Yamaguchi(2007)_5	0.8704391	2.036158
Eckart(1951)	1.192608	4.979783
Yamaguchi(2007)_6	1.210178	5.25878
Yamaguchi(2007)_3	1.488983	3.147347
Iwagaki(2007)	1.488983	3.147347
Yamaguchi(2007)_4	1.587265	3.177284

8.3 Location of maximum error

For reproducibility, the following table gives the sampled input associated with each maximum. The reference row has no meaningful worst-case location because its error against itself is zero throughout the grid.

Method	Input at maximum error
Pade(2025)_11	0.0001000
Pade(2025)_8	0.0001000
Pade(2025)_12	0.0001000

Method	Input at maximum error
Pade(2025)_13	0.0001000
Pade(2025)_9	0.0001000
Pade(2025)_10	0.0001000
Pade(2025)_7	5.3305743
Pade(2025)_6	0.7685982
Carvalho(2025)_1	0.3255964
Simarro&Orfila(2013)	0.0359172
Pade(2025)_4	0.0001000
Pade(2025)_5	2.9490468
Vatankhah(2013)_1	0.9514543
Pade(2025)_3	0.0001000
Carvalho(2025)_2	1.2002893
Hunt(1979)_9	3.7891793
Fenton&McKee(1990)_1	0.4393316
Pade(2025)_2	0.0001000
Wu&Thornton(1986)	0.7089029
Vatankhah(2013)_2	0.0704776
Carvalho(2025)_3	1.1273982
Yamaguchi(2007)_10	1.1273982
Carvalho(2025)_4	0.3463326
Beji(2013)	0.2998331
Carvalho(2025)_5	1.5603461
Carvalho(2025)_6	0.5769449
Carvalho(2025)_7	1.3862872
Hunt(1979)_5	1.8098095
Nielsen(1982)	1.9995777
You(2002)	1.9995777
Carvalho(2025)_8	2.6549690
Carvalho(2025)_9	2.6568541
Carvalho(2025)_10	0.3940888
Carvalho(2025)_11	0.3871768
Yu(2014)	0.0843018
Yamaguchi(2007)_9	1.6590004
Carvalho(2025)_12	0.9621366
Pade(2025)_1	0.0001000
Carvalho(2025)_13	0.0007284
Yamaguchi(2007)_2	0.1791858
Carvalho(2025)_14	0.2200300
Gilbert(2000)	0.4858311
Yamaguchi(2007)_8	1.0287439
Guo(2002)	1.7821612
Carvalho(2025)_15	0.1421119
Carvalho(2025)_16	0.1364566
Guan&Ju(2005)	3.5937559
Carvalho(2025)_17	0.3281099
Yamaguchi(2007)_7	1.6847636

Method	Input at maximum error
Carvalho(2025)_18	1.4912253
Carvalho(2025)_19	0.3067452
Carvalho(2025)_20	1.1160875
Fenton&McKee(1990)_2	0.3400489
Yamaguchi(2007)_1	1.9813549
Yamaguchi(2007)_5	1.8695048
Eckart(1951)	0.6938220
Yamaguchi(2007)_6	0.7403215
Yamaguchi(2007)_3	1.8060393
Iwagaki(2007)	1.8060393
Yamaguchi(2007)_4	0.4267642

8.4 Interpreting the comparison

The source's ordinary output uses seven decimal places for percentage errors. A displayed value of 0.0000000% can therefore represent a small nonzero value. In particular, the high-order fractional rational fits should not be described as numerically identical to the reference solely because of that formatting.

Some registered methods duplicate one another algebraically. In this source, the Nielsen and You functions coincide, Iwagaki matches Yamaguchi formula 3, and Yamaguchi formula 10 matches Carvalho formula 3. Their inclusion preserves the source catalogue, but they are not independent algorithmic discoveries within this implementation.

A fixed Newton correction can considerably reduce seed error without the cost of a full convergence loop. This explains the small errors of the Simarro–Orfila, Fenton–McKee corrected, and Carvalho formula 1 methods. Nevertheless, the accuracy remains dependent on the seed, input interval and floating-point evaluation.

The fractional rational family provides very small sampled errors at high order, while formula 5 within that family illustrates that increasing degree need not improve maximum error monotonically. The expressions contain fitted coefficients, and denominator behaviour matters as much as degree.

8.5 Timing method and reproducibility

For each registered method, `main()` draws one random scalar input in the comparison interval, evaluates the function 1000 times, measures elapsed time using `time.perf_counter()`, and multiplies that elapsed time by 1000. Thus:

$$t_{1M} = 1000\, t_{1000}$$

`Time1M` is an extrapolated estimate of seconds per million calls. It is not the measured duration of a million-call run. Each method receives a separately drawn random input, and no random seed is set by the driver. The result is affected by input, Python version, machine load, processor and mathematical library.

For a reproducible timing study, use a common input set, fixed random seed where relevant,

repeated trials and a summary of trial variability. Record source revision, software versions and hardware. Algebraic operation counts alone cannot establish actual speed, and scalar Python measurements do not describe GPU throughput or native NumPy vectorization.

8.6 Choosing an approximation

Selection should begin with the physical model and the input domain, followed by an error requirement for the downstream calculation. Use the numerical reference to verify representative cases. A compact approximation may be adequate for exploratory work, while long propagation distances or sensitive derivative calculations may require a corrected method or iterative inversion.

Carvalho formula 5 is convenient when a short direct wavelength expression is desired, but its full coefficients should be retained. The fractional rational formulas are appropriate for comparison within their documented interval; they should not be treated as globally valid deep-water substitutes. Piecewise methods require attention at branch boundaries. For all methods, distinguish approximation error from uncertainty in period, depth and the applicability of linear wave theory.

9. Software installation and usage

9.1 Files and dependencies

Save the source under its repository filename, `wave-disp-equation.py`. Place any example scripts in the same directory.

A working Python 3 installation, NumPy and Matplotlib are required. `math`, `time` and `importlib` belong to Python's standard library. No minimum Python version or pinned dependency versions are declared in the implementation; use a supported Python 3 environment compatible with your NumPy and Matplotlib installation.

9.2 Windows PowerShell

Open PowerShell in the directory containing the source. These commands use the virtual environment directly, so activation is unnecessary:

```
py -3 -m venv .venv
.\.venv\Scripts\python.exe -m pip install --upgrade pip
.\.venv\Scripts\python.exe -m pip install numpy matplotlib
.\.venv\Scripts\python.exe .\wave-disp-equation.py
```

9.3 Linux and macOS

Open a terminal in the directory containing the source:

```
python3 -m venv .venv
.venv/bin/python -m pip install --upgrade pip
.venv/bin/python -m pip install numpy matplotlib
.venv/bin/python wave-disp-equation.py
```

9.4 Run and interpret the benchmark

Running the script evaluates all registered methods, prints the ranking, writes a text file, saves a PNG and then calls `plt.show()`. On an interactive graphical backend, the terminal may

wait until the plot window is closed. Output files are written to the **current working directory**, and existing files with the same names are overwritten. The files are `wave-disp-equation_output.txt` and `wave-disp-equation_errors.png`.

Output	Contents
Text file	Errors and timing estimates
PNG file	Mean errors, 300 dpi
Terminal	Ranking and method errors

Column	Interpretation
Rank	Order by mean error, then maximum error
Method	Key in `ordered_approx`
AvgErr	Mean absolute relative error, percent
MaxErr	Largest sampled absolute relative error, percent
k0h_MaxErr	First sample attaining the largest error
Time1M	Estimated seconds per million scalar calls

The plot compares mean errors only. It does not show maximum errors, signed errors or timing. Errors printed as zero are rounded to the output precision. If error evaluation fails for a method, its row is marked `ERROR` and sorted after valid rows.

9.5 Run without a graphical display

Select the non-interactive Matplotlib backend before launching the script. The text and PNG files are still generated.

Windows PowerShell:

```
$env:MPLBACKEND = "Agg"
.\.venv\Scripts\python.exe .\wave-disp-equation.py
```

Linux or macOS:

```
MPLBACKEND=Agg .venv/bin/python wave-disp-equation.py
```

9.6 Import the functions

A hyphen is not valid in an ordinary Python module identifier. Load the file explicitly with `importlib` rather than writing an ordinary import statement using its filename. Save this example as `example.py` beside the source:

```
from pathlib import Path
import importlib.util

source = Path(__file__).with_name("wave-disp-equation.py")
spec = importlib.util.spec_from_file_location("wave_disp", source)
wave_disp = importlib.util.module_from_spec(spec)
spec.loader.exec_module(wave_disp)

alpha = 0.4
```

```
beta_ref = wave_disp.kh_numeric(alpha)
beta_approx = wave_disp.carvalho2025(alpha, 5)
print(f"Input k0h:   {alpha:.9f}")
print(f"Reference kh: {beta_ref:.9f}")
print(f"Carvalho 5:   {beta_approx:.9f}")
```

Importing does not execute the benchmark because the source protects `main()` with a main-module guard. NumPy and Matplotlib are nevertheless imported, so both dependencies remain required even when only one solver function is used.

9.7 Dimensional worked example

For a period of 9 s and a water depth of 5 m, use the still-water depth at the water level relevant to the calculation. Depth is positive and is not the seabed elevation relative to a datum.

$$\alpha = \frac{4\pi^2 h}{gT^2}, \qquad k = \frac{\beta}{h}, \qquad L = \frac{2\pi h}{\beta}$$

$$L_0 = \frac{gT^2}{2\pi}, \qquad L_{a,5} = L_0 \tanh\left(1.199315^{\alpha^{1.047086}} \alpha^{0.499947}\right)$$

After loading `wave_disp` as above, append:

```
import math

g = 9.80665
T = 9.0
h = 5.0
if T <= 0 or h <= 0:
    raise ValueError("Period and water depth must be positive.")

omega = 2.0 * math.pi / T
alpha = omega**2 * h / g
beta = wave_disp.kh_numeric(alpha)
beta5 = wave_disp.carvalho2025(alpha, 5)
k = beta / h
L = 2.0 * math.pi / k
L5 = 2.0 * math.pi * h / beta5
cp = omega / k
cg = 0.5 * cp * (1.0 + 2.0 * beta / math.sinh(2.0 * beta))
residual = beta * math.tanh(beta) - alpha

for name, value in [
    ("k0h", alpha), ("kh", beta), ("k (rad/m)", k),
    ("L (m)", L), ("Carvalho 5 L (m)", L5),
    ("Phase speed (m/s)", cp), ("Group speed (m/s)", cg),
    ("Dimensionless residual", residual),
]:
    print(f"{name:25s} {value:.9g}")
```

Expected values, rounded from the implementation:

Quantity	Value
Input k0h	0.248498657
Numerical kh	0.520107668
Wavenumber (rad/m)	0.104021534
Numerical wavelength (m)	60.402736704
Carvalho 5 wavelength (m)	60.399342429
Phase speed (m/s)	6.711415189
Group speed (m/s)	6.174792735

The two wavelength estimates differ because Carvalho(2025)_5 is an approximation. Keep its full coefficients when copying the expression; replacing them with three-decimal versions changes the fitted formula.

9.8 Function reference

All solver inputs and outputs are dimensionless. `formula` is an integer selector, not a depth or a period.

Function or object	Use
`kh_numeric`	Newton reference
`carvalho2025`	Selectors 1–20
`pade2025`	Selectors 1–13
`YamaguchiNonaka`	Selectors 1–10
`ordered_approx`	Named scalar callables
`compute_errors`	Mean, maximum and location
`main()`	Benchmark and file outputs

Family signatures are `carvalho2025(k0h, formula)`, `pade2025(k0h, formula)` and `YamaguchiNonaka(k0h, formula)`. The error helper is `compute_errors(func, k0h_vals)`. The numerical reference accepts `kh_numeric(k0h, tol=1e-15, max_iter=100)`.

The remaining standalone solver names are:

```
Simarro_2013
vatankhah2013_1       vatankhah2013_2
hunt1979_9           hunt1979_5
fenton_mckee1990_1    fenton_mckee1990_2
wu_thornton1986      beji2013
nielsen1982          you2002
yu2014               gilbert2000
guo2002              guan2005
iwagaki2007          eckart1951
```

Each of these takes one scalar `k0h` argument. Selectors outside the implemented family ranges return `-1`, which is an invalid sentinel rather than a physical result. Inputs are not comprehensively validated by the source.

9.9 Calculate a sequence or a custom error grid

The functions use scalar `math` operations; passing a NumPy array directly is generally unsupported. A list comprehension calls each scalar solver explicitly:

```
import numpy as np

alpha_values = np.linspace(0.0001, 2.0 * np.pi, 10000)
beta_values = np.array([
    wave_disp.carvalho2025(float(a), 5) for a in alpha_values
])
mean_error, max_error, location = wave_disp.compute_errors(
    lambda a: wave_disp.carvalho2025(float(a), 5),
    alpha_values,
)
print(mean_error, max_error, location)
```

To change the grid for the full benchmark, edit the `k0h_vals` assignment inside `main()`. There are no built-in `--min`, `--max` or `--step` options. To change the included methods, edit `ordered_approx` or construct a separate subset in an importing script. Extending a numerical sweep beyond the default interval does not extend a fitted formula's validated domain.

9.10 Input limits and failure interpretation

Use finite, positive inputs for general comparisons. `kh_numeric(0)` returns zero, and several standalone routines also handle zero explicitly, but the Carvalho family does not provide a common zero-input guard. Negative depths, zero periods and non-finite values are not physical inputs to the dimensional example.

At large positive input, intermediate exponentials, powers, `sinh` or `cosh` can overflow even when the underlying mathematical solution remains finite. Fractional rational expressions may also become poorly conditioned or approach denominator zeros outside their fitted interval. The source does not enforce the documented Padé interval.

A `ModuleNotFoundError` for NumPy or Matplotlib usually means the script is being run with a different interpreter from the one used to install the packages. Use the same explicit virtual-environment Python path for installation and execution. A plot window remaining open is normal after `plt.show()`; use the non-interactive commands when only file outputs are required.

The driver catches error-evaluation and timing exceptions separately. A timing failure can leave a missing timing value that the later numeric formatting does not handle. Numerical routines also do not consistently reject NaN values. These limitations should be considered when integrating the functions into a larger calculation workflow.

10. Additional approximations for restricted regimes

The expressions in this section are theoretical background presented alongside the implemented catalogue. They are not additional separately registered functions in the implementation. Where a similar branch appears in a registered method, the complete implemented definition and its switching condition are given in Section 4.

10.1 Shallow-water polynomial approximations

A first empirical shallow-water expression attributed to Nielsen in the literature comparison is:

$$\beta_{N1} = \sqrt{\alpha}\left(1 + \frac{5\alpha}{8\pi}\right)$$

A second expression retains two powers inside the shallow-water correction:

$$\beta_{N2} = \sqrt{\alpha}\left(1 + \frac{\alpha}{6} + \frac{11\alpha^2}{360}\right)$$

The Wu–Thornton shallow expression is:

$$\beta_{WT,s} = \sqrt{\alpha}\left[1 + \frac{\alpha}{6}\left(1 + \frac{\alpha}{5}\right)\right]$$

These are local approximations rather than deep-water asymptotic formulas. The presence of the leading square root does not justify extrapolating the correction polynomial to arbitrarily large input.

10.2 Venezian rational expressions

The first background expression is:

$$\beta_{Ve1} = \frac{\sqrt{\alpha}}{1 - \alpha/6}$$

It has an explicit denominator zero at $\alpha = 6$, illustrating why an intended local approximation needs a domain restriction.

The second expression is:

$$\beta_{Ve2} = \sqrt{\alpha}\,\frac{N_V(\alpha)}{D_V(\alpha)}$$

$$N_V(\alpha) = 1 - 0.428868\alpha + 0.093928\alpha^2 - 0.002694\alpha^3$$

$$D_V(\alpha) = 1 - 0.595534\alpha + 0.162628\alpha^2 - 0.014975\alpha^3$$

10.3 Higher-order shallow expansion

The Olson/You expression quoted in the literature comparison has the form:

$$\beta_O = \sqrt{\alpha}\,[P_O(\alpha)]^{-1/2}, \qquad P_O(\alpha) = \sum_{j=0}^{7} p_j \alpha^j$$

Its complete coefficients are:

j	p_j
0	1

j	p_j
1	$-1/3$
2	$1/45$
3	$1/189$
4	0.000776
5	-0.000044
6	-0.000071
7	-0.000022

10.4 Deep-water exponential expansions

A leading correction attributed to Nielsen is:

$$\beta_{N,d} = \alpha[1 + 2\exp(-2\alpha)]$$

The Wu–Thornton background expression is:

$$q = \exp\{-2\alpha[1 + 1.26\exp(-1.84\alpha)]\}$$

$$\beta_{WT,d} = \alpha[1 + 2q(1 + q)]$$

The latter is algebraically the deep branch of the implemented Wu–Thornton function. These exponential corrections become appropriate as the hyperbolic tangent approaches its deep-water limit; they are not accurate shallow-water expansions.

10.5 Historical ranges quoted in the literature comparison

The following limits and signed error summaries describe the restricted-domain literature comparison. They have not been independently reproduced as part of the registered-method benchmark. The variable in these ranges is $h/L_0 = \alpha/(2\pi)$, not h/L.

Background formula	Quoted range
Nielsen 1	$h/L_0 \leq 0.192$
Nielsen 2	$h/L_0 \leq 0.401$
Venezian 1	$h/L_0 \leq 0.165$
Wu–Thornton shallow	$h/L_0 \leq 0.219$
Olson/You	$h/L_0 \leq 0.186$
Venezian 2	$h/L_0 \leq 0.159$
Nielsen deep	$h/L_0 \geq 0.300$
Wu–Thornton deep	$h/L_0 \geq 0.195$

Background formula	Quoted error (%)
Nielsen 1	± 0.74
Nielsen 2	± 0.44

Background formula	Quoted error (%)
Venezian 1	±0.048
Olson/You	±0.00003
Venezian 2	-0.0002 to +0.000006
Nielsen deep	-0.55
Wu–Thornton deep	±0.025

Consult the original publications before applying these historical bounds. The software neither validates these ranges nor exposes all of these expressions as callable methods.

11. Extensions of the physical model

11.1 Waves on a uniform current

For a uniform current vector V, the intrinsic frequency is the frequency measured in a frame moving with the fluid:

$$\sigma = \omega - \mathrm{k} \cdot \mathrm{V}$$

The linear relation is then:

$$(\omega - \mathrm{k} \cdot \mathrm{V})^2 = gk \tanh(kh)$$

$$\sigma^2 = gk \tanh(kh)$$

When absolute frequency and current are prescribed, the intrinsic frequency depends on the unknown wavenumber. The no-current inversion cannot simply be used once with the absolute period. Opposing currents can reduce absolute group propagation and can cause wave blocking; rotational or vertically sheared currents require additional analysis.

11.2 Gravity-capillary waves

For surface tension γ, the restoring force includes a cubic wavenumber term:

$$\omega^2 = \left(gk + \frac{\gamma}{\rho} k^3 \right) \tanh(kh)$$

The relative importance of capillarity depends on density, gravity, surface tension and wavelength. The gravity-only formulas in Sections 3–7 do not invert this relation.

11.3 Finite-amplitude dispersion

In deep water, a leading Stokes amplitude correction can be written as:

$$\omega^2 \approx gk[1 + (ka)^2]$$

Unlike the linear relation, it depends on wave amplitude. At fixed wavenumber, the leading correction increases phase speed as amplitude grows. Finite-depth nonlinear waves require an appropriate nonlinear theory; this single deep-water expression is not a general substitute. Modulational instability also requires a wave-interaction analysis beyond the dispersion correction itself.

11.4 Interfacial waves

For two semi-infinite, homogeneous fluid layers with lower density ρ greater than upper density ρ', the deep-water interfacial relation is:

$$\omega^2 = gk\frac{\rho - \rho'}{\rho + \rho'}$$

The density contrast reduces the effective restoring force relative to a free-surface gravity wave. Finite layer thickness and continuous stratification change the relation. The software does not solve these stratified-flow problems.

12. Conclusions

The linear wave-dispersion equation has one positive dimensionless root for each positive input. Numerical iteration offers a reference solution, while explicit formulas provide alternatives with different costs, input restrictions and error distributions. A useful comparison must preserve the exact implemented coefficients, state the sampling interval, distinguish mean from maximum error, and avoid interpreting rounded zeros as exact identity.

The complete catalogue shows three recurring strategies: direct matching of shallow- and deep-water behaviour, fitted expressions or rational functions over a finite domain, and a fixed Newton correction applied to an accurate seed. Several registered methods are algebraically identical, while the fractional rational family achieves very small errors on its fitted interval without possessing the correct unrestricted deep-water asymptote.

For practical use, period and depth must first be converted to the dimensionless input, and the calculated output must then be converted back to wavelength or wavenumber. Numerical accuracy should be selected in relation to the physical model and the sensitivity of the intended calculation. The software is a transparent comparison tool and a collection of scalar formulas; it is not a replacement for assessing the applicability of linear wave theory.

13. References

The bibliography records the sources associated with the implemented formulas. Code labels are retained even when they refer to a comparison year rather than the original publication year. The two Hunt titles and two Yamaguchi entries are retained as bibliographic variants from the literature comparison and should not be interpreted as independent verification. The Carvalho (2025) families are repository contributions; a peer-reviewed publication for these exact coefficient sets is not identified here.

1. **Wikipedia**. “Airy wave theory”. Wikipedia.
2. **Wikipedia**. “Dispersion (water waves)”. Wikipedia.

3. **Yu, J.** (2014). "A Note on Approximations of the Dispersion Relationship of Water Waves", *Journal of Engineering Mechanics (ASCE)*, 140(1), 233–237.

4. **Simarro, G. & Orfila, A.** (2013). "Improved explicit approximation of linear dispersion relationship for gravity waves: Another discussion", *Coastal Engineering*, 80, 15–16.

5. **Vatankhah, A.R. & Aghashariatmadari, Z.** (2013). "Improved explicit approximation of linear dispersion relationship for gravity waves: a discussion", *Coastal Engineering*, 78, 21–22.

6. **Beji, S.** (2013). "Improved explicit approximation of linear dispersion relationship for gravity waves", *Coastal Engineering*, 73, 11–12.

7. **You, Z.J.** (2008). "A close approximation of wave dispersion relation for direct calculations", *Applied Ocean Research*, 30(2), 141–143.

8. **Yamaguchi, M. & Nonaka, H.** (2007). "Comparative Study of Explicit Solutions to Wave Dispersion Equation", *Journal of JSCE (Ocean Engineering)*, 63(1), 53–66.

9. **Yamaguchi, M. and H. Nonaka.** Comparative study of explicit solutions to wave dispersion equation, *Annu. Jour. Eng.*, Ehime Univ., Vol. 6, 2007 in CD-ROM.

10. **Carvalho, R.** (2006). Unpublished work based on gene expression programming for wave dispersion equations.

11. **You, Z.J.** "Discussion of 'Simple and explicit solution to the wave dispersion equation' ", *Coastal Engineering 45 (2002) 71-74*, Coastal Eng., Vol. 48, pp. 133-135, 2003.

12. **Guo, J.** (2002). "Simple and explicit solution of the wave dispersion equation", *Coastal Engineering*, 45, 71–74.

13. **Fenton, J.D. & McKee, W.D.** (1990). "On calculating the lengths of water waves", *Coastal Engineering*, 14, 499–513.

14. **Fenton, J.D.** "The numerical solution of steady water wave problems", *Computers & Geosciences*, Vol. 14, No. 3, pp. 357–368, 1988.

15. **Fenton, J.D.** (1972). "A ninth-order solution for the solitary wave", *Journal of Fluid Mechanics*, 53, 257–271.

16. **Wu, C. S. and E. B. Thornton.** "Wave numbers of linear progressive waves", *Journal of Waterway, Port, Coastal and Ocean Engineering*, ASCE, Vol. 112, No. 4, pp. 536–540, 1986.

17. **Nielsen, P.** "Explicit solutions to practical wave problems", *Proc. 19th ICCE*, Vol. 1, pp. 968–982, 1984.

18. **Nielsen, P.** "Explicit formulae for practical wave calculations", *Coastal Eng.*, No. 6, pp. 389–398, 1982.

19. **Hunt, J.N.** (1979). "A simple approximation for the dispersion relation of water waves", *Journal of Waterway, Port, Coastal and Ocean Engineering*, 105(4), 457–459.

20. **Hunt, J. N.** "Direct solution of wave dispersion equation", *J. Waterway, Port, Coastal and Ocean Div.*, Proc. ASCE, Vol. 105, No. WW4, pp. 457–459, 1979.

21. **Eckart, C.** (1951). "The propagation of gravity waves from deep to shallow water", U.S. Department of Commerce, National Bureau of Standards Circular 521.

22. **Carvalho, R.** (2025). Explicit and fractional rational approximation families implemented in wave-disp-equation.py.

23. **Guan, C. & Ju, H.** (2005). "Empirical formula for wavelength of ocean wave in finite depth water", *Chinese Journal of Oceanology and Limnology*, 23(1), 17–22.

24. **Ferreira, C.** (2006). *Gene Expression Programming: Mathematical Modeling by an Artificial Intelligence*, 2nd edition. Springer. Background on GEP; this is not a derivation of the fitted coefficients listed above.